

Getting Started with Python Testing Using Pytest

Pytest로 시작하는 파이썬 테스트

예외 처리와 경계값 테스트

예외 처리



테스트의 진짜 목적은 '성공'을 확인하는 것이 아님



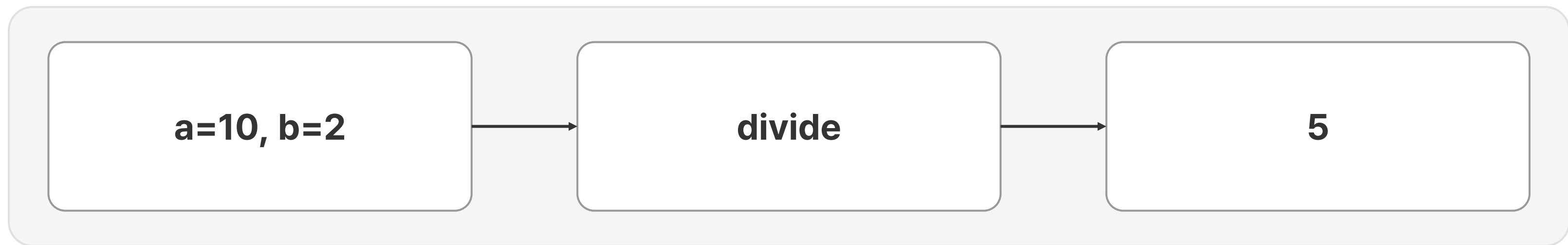
코드가 작동하는가?



- 테스트는 코드가 **올바르게 실패**하는지 검증
- 테스트는 미래의 코드 수정을 위한 **안전망**
- 테스트는 더 나은 소프트웨어 구조를 강제하는 **설계 도구**

정상 케이스만 확인하는 것은 절반의 테스트

정상 동작 검증



경계값 및 예외 상황 검증

'잘못된 입력에서 올바르게 실패하는가?'를 반드시 질문해야 함



테스트 코드 작성을 지탱하는 두 개의 기둥



기본 테스트 (Basic Test)

명칭 규칙 준수와
결과값의 단순 검증

`test_, assert`



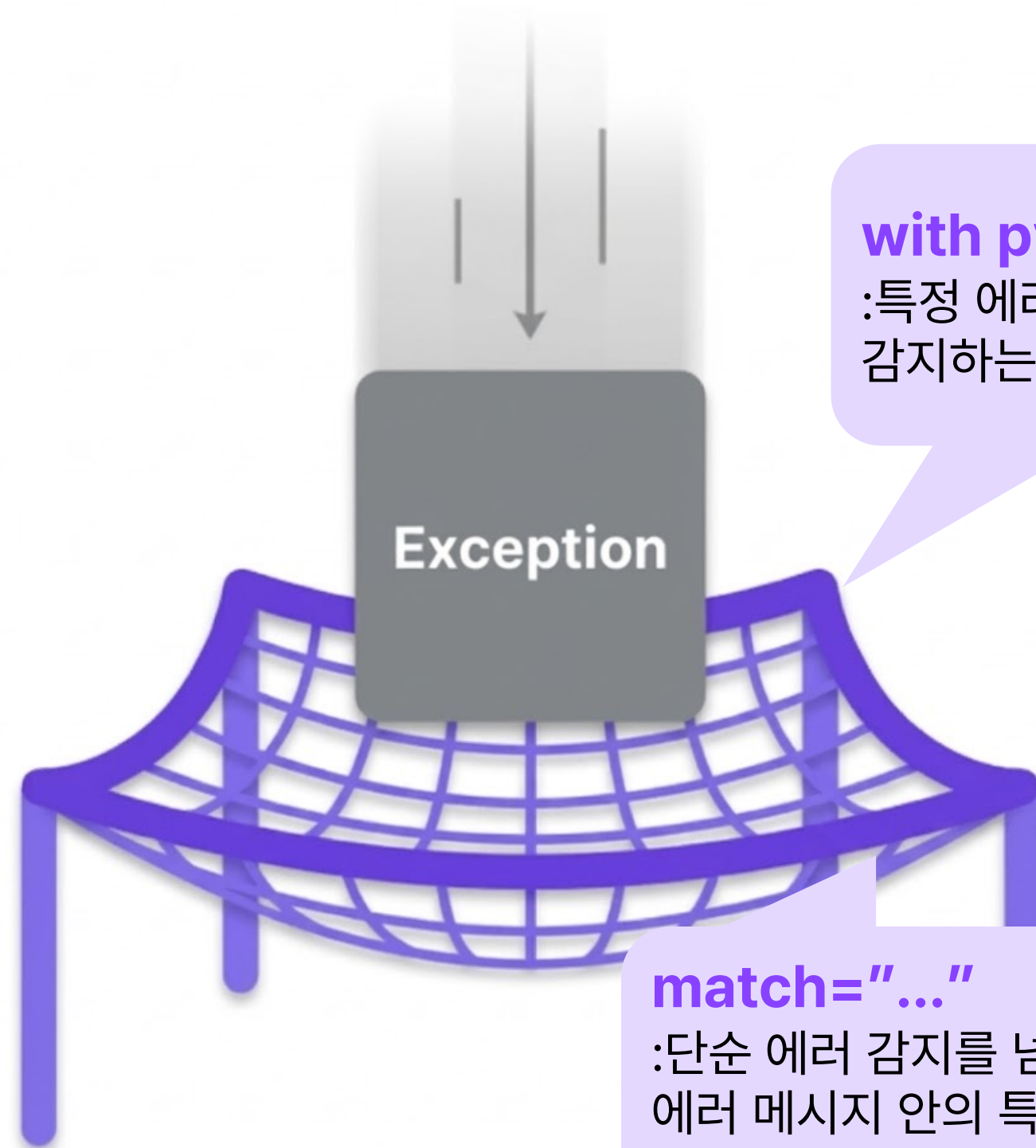
예외 테스트 (Exception Test)

의도적인 에러 유발과
정밀한 메시지 검증

`pytest.raises, match`



2단계 : 의도된 에러를 낚아채다 (예외 처리)



with pytest.raises()
:특정 에러가 발생하는지
감지하는 안전망 구축

match="..."
:단순 에러 감지를 넘어,
에러 메시지 안의 특정
문자열까지 정밀 검사

```
# 0으로 나누는 상황을 고의로 발생시키고 검증  
with pytest.raises(ValueError, match="0으로  
나눌 수 없습니다"):  
    divide(10, 0)
```


예상된 실패를 잡아내는 '포수 미트'

특정 코드를 실행했을 때, 의도한 예외가 발생하는지 명시적으로 검증합니다.

```
def test_divide_by_zero():  
    with pytest.raises(ValueError):  
        divide(10, 0)
```

이 블록 안에서 ValueError가
발생해야만 테스트가 성공함

예외를 유발하는 실제 함수 호출

Reference (calculator.py)

```
def divide(a, b):  
    if b == 0: raise ValueError(...)  
    return a / b
```

정밀 검증 - 에러의 '원인'까지 확인하기

단순히 에러의 종류(ValueError)만 포착하는 것이 아니라,
match 인자를 통해 발생한 에러 메시지가 특정 문자열을 포함하는지 필터링함

```
with pytest.raises(ValueError, match="0으로 나눌 수 없습니다"):
    calculate(10, 0)
```



이 문자열이 실제 에러 메시지에
포함되어 있어야만 테스트가 통과됨



예외 검증 : pytest.raises

```
import pytest

def test_zero_division():
    with pytest.raises(ZeroDivisionError):
        1 / 0

def test_match_msg():
    with pytest.raises(ValueError, match="invalid"):
        int("abc")

def test_exc_info():
    with pytest.raises(TypeError) as exc:
        "a" + 1
    assert "str" in str(exc.value)
# 블록 안에서 예외 발생 → 테스트 통과
```

```
pytest.raises(
    expected_exception,
    *, match=None
)
```

expected_exception : 발생을 기대하는 예외 클래스

match : 에러 메시지를 정규표현식으로 검증

as exc : 예외 객체를 변수에 저장

동작 방식

- 블록 안에서 지정한 예외 발생 → **테스트 통과**
- 예외가 발생하지 않으면 → **테스트 실패**
- 다른 예외가 발생하면 → 그 예외가 그대로 발생하여 **테스트 실패**



Pytest 핵심 문법 치트시트 (Cheat Sheet)



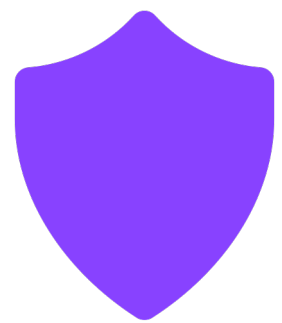
test_

파일명, 함수명은 반드시 test_로 시작
(클래스는 Test)



assert

복잡한 메서드 없이 표준
연산자(==, !=, <, in)로 결과 검증



pytest.raises

with 구문과 함께 사용하여 의도된
에러 발생 여부 확인



match

raises 안에 인자로 추가하여 에러
메시지 내용까지 정밀 필터링